



Com crear la teva pròpia Autoritat de Certificació
i emetre certificats SSL autofirmats

 SSL / TLS

 PKI

 OpenSSL

 FP Grau Mitjà

BLOC 1 · TEORIA



SSL - Secure Sockets Layer

Protocol **ja obsolet**, desenvolupat per Netscape als anys 90. El nom "SSL" continua usant-se col·loquialment però en la pràctica és deprecad.



TLS - Transport Layer Security

El successor modern i segur de SSL. Versió actual: **TLS 1.3** (2018). És el que realment usa HTTPS avui en dia.



Quan algú diu "certificat SSL" en realitat parla d'un **certificat TLS**. Són el mateix en la pràctica diària. TLS xifra la comunicació entre client i servidor per garantir **confidencialitat**, **integritat** i **autenticació**.

1

Xifrat de la comunicació

Ningú pot llegir les dades en trànsit entre el navegador i el servidor, tot i interceptar el tràfic de xarxa.

2

Autenticació de la identitat

El certificat prova que el servidor és qui diu ser. Sense això, qualsevol podria suplantar el teu banc.

3

Integritat de les dades

TLS inclou signatura criptogràfica per detectar si les dades han estat manipulades durant la transmissió.



Un cadenat  al navegador només indica que la connexió està xifrada. **No garanteix que el lloc sigui legítim.** Els atacants també poden usar HTTPS.

BLOC 1 · TEORIA



PKI - Public Key Infrastructure

Sistema de polítiques, rols i procediments que gestiona certificats digitals i claus criptogràfiques. Dóna confiança a l'ecosistema.



CA - Autoritat de Certificació

Entitat de confiança que **emet i signa** certificats digitals. Pot ser pública (DigiCert, Let's Encrypt) o privada (la teva pròpia).

 **CA Arrel**
rootCA.crt

→ signa →

 **Certificat**
Servidor
server.crt

→ usa →

 **Servei HTTPS**
Apache / Nginx

La CA "avala" l'autenticitat del certificat del servidor amb la seva signatura digital

CA Pública (Let's Encrypt, DigiCert...)

- ✓ **Confiança universal**: tots els navegadors la reconeixen
- ✓ Ideal per a serveis públics a internet
- ✓ Let's Encrypt és **gratuïta** i automatitzable
- ✗ Necessita accés a internet i DNS públic
- ✗ Sense control sobre qui l'usa

CA Pròpia (autofirmada)

- ✓ **Control total**: tu decideixes qui rep certificats
- ✓ Funciona sense internet (xarxes internes, laboratori)
- ✓ Sense caducitat forçada anual a la CA arrel
- ✓ Perfecta per a **entorns de test**
- ✗ Cal instal·lar la CA manualment a cada client
- ✗ No vàlida per a serveis públics



Tests i desenvolupament local

Provar una aplicació web amb HTTPS al teu portàtil o xarxa local, sense necessitar un domini públic.



Sense accés a internet

Xarxes aïllades, entorns industrials, laboratoris on no hi ha connectivitat exterior.



Control de l'emissió

Tu decideixes quins serveis i servidors reben certificats a la teva organització.



Serveis de llarga durada

Evitar renovacions anuals costoses en sistemes interns que no necessiten validació pública.



No usis mai certificats autofirmats en serveis públics on la identitat hagi de ser verificable per tercers. Només xifren la connexió però no proven qui ets.

BLOC 1 · TEORIA



Clau privada

Fitxer secret que **mai has de compartir**. S'usa per firmar i desxifrar. Tipus RSA de 4096 bits en el nostre cas.



Clau pública

Es comparteix lliurement. Qualsevol pot xifrar amb ella, però només el propietari de la privada pot desxifrar.



CSR

Certificate Signing Request.

Petició que enviem a la CA amb la nostra clau pública perquè ens emeti un certificat.

 **Clau Privada**
(guarda en secret)



 **CSR**
(petició a la CA)



 **Certificat**
(fitxer .crt)



La navalla suïssa de la criptografia. Eina de línia d'ordres open-source per gestionar certificats, claus i TLS.



Instal·lació a Ubuntu/Debian:

```
sudo apt install openssl  
  
# Verificar la versió instal·lada  
openssl version
```

genrsa

Genera claus privades RSA

req

Gestiona CSRs i certificats x509

x509

Signa i gestiona certificats

⚙️ BLOC 2 · PRÀCTICA GUIADA



Pas 1 & 2
Crear CA

Clau privada CA + Certificat CA arrel



Pas 3 & 4
Preparar servidor

Clau privada servidor + CSR



Pas 5 & 6
Emetre certificat

Signar CSR amb CA + (opcional) bundle .pem



Crea un directori de treball abans de començar: `mkdir ~/els-meus-certs && cd ~/els-meus-certs`

Generem la clau privada RSA de 4096 bits de la nostra Autoritat de Certificació

```
openssl genrsa -out rootCA.key 4096
```

genrsa

Algorisme RSA per
generar el parell de claus

-out rootCA.key

Fitxer on es guarda la clau
privada

4096

Longitud de la clau en bits.
Més bits = més segur (i
més lent)



rootCA.key és el fitxer més sensible del procés. Qui tingui aquest fitxer pot emetre certificats falsos signats per la teva CA. Protegeix-lo amb permisos restrictius: `chmod 400 rootCA.key`



La CA es signa a si mateixa (self-signed). Genera el certificat arrel amb validesa de 1024 dies.

```
openssl req -x509 -new -nodes \  
  -key rootCA.key \  
  -days 1024 \  
  -out rootCA.crt
```

Opcions utilitzades

- **-x509** — Genera directament un certificat (no un CSR)
- **-new** — Nova petició
- **-nodes** — Sense contrasenya a la clau
- **-days 1024** — Validesa (~2,8 anys)

Et demanarà aquestes dades

- **Country Name (C):** ES
- **State (ST):** Catalunya
- **City (L):** Barcelona
- **Organization (O):** La Meva Empresa
- **Common Name (CN):** La Meva CA Arrel

⚙️ PAS 3 DE 6

El mateix procés que per a la CA, però ara per al servidor o servei que volem protegir

```
openssl genrsa -out server.key 4096
```



En aquest punt hauries de tenir al teu directori de treball:

-  `rootCA.key` — Clau privada de la CA (guarda-la bé!)
-  `rootCA.crt` — Certificat de la CA (pots distribuir-lo)
-  `server.key` — Clau privada del servidor



Pots generar certificats per a múltiples servidors repetint els passos 3, 4 i 5 amb noms de fitxer diferents. La CA arrel es crea **una sola vegada**.

La petició de signatura. Conté la clau pública del servidor i les dades d'identitat.

```
openssl req -new \  
-key server.key \  
-out server.csr
```



El camp més important aquí és el Common Name (CN).

Has de posar el **FQDN** (nom de domini complet) del servidor. Per exemple: `miservidor.local` o

`www.lamivaempresa.com`

Si el CN no coincideix amb el domini al qual accedeixes, el navegador donarà error de certificat.



Pots inspeccionar el contingut del CSR per verificar que tot és correcte:

```
openssl req -text -noout -in server.csr
```

La CA "avala" el CSR del servidor i emet el certificat signat. Validesa: 500 dies.

```
openssl x509 -req \  
  -in      server.csr \  
  -CA      rootCA.crt \  
  -CAkey    rootCA.key \  
  -CAcreateserial \  
  -days    500 \  
  -out      server.crt
```

Paràmetres clau

- `-in server.csr` — El CSR que signarem
- `-CA rootCA.crt` — Certificat de la nostra CA
- `-CAkey rootCA.key` — Clau privada de la CA
- `-CAcreateserial` — Crea un número de sèrie únic
- `-days 500` — Validesa del certificat

Resultat: server.crt

- ✓ **Certificat vàlid** signat per la teva CA
- ✓ Llest per usar-se a Apache, Nginx, etc.
- 📄 Conté la clau pública del servidor
- 📄 Signat digitalment per rootCA.key

⚙️ PAS 6 DE 6 · OPCIONAL

Alguns serveis requereixen tots els certificats i la clau en un sol fitxer `.pem`

```
# L'ordre importa: primer cert servidor, després clau, després CA
cat server.crt >> certComplet.pem
cat server.key >> certComplet.pem
cat rootCA.crt >> certComplet.pem
```



L'ordre de concatenació és crític. Si s'inverteix, el servei pot fallar en carregar el certificat. Sempre: certificat → clau → CA (o cadena de CAs).



El fitxer `.pem` és habitual a **HAProxy**, **Dovecot**, **Postfix** i altres serveis de xarxa. Apache i Nginx solen preferir fitxers separats.

⚙️ BLOC 2 · RESUM



rootCA.key

Clau privada de la CA ·

SECRET — mai compartir



rootCA.crt

Certificat de la CA arrel · S'instal·la als clients perquè confiïn en els teus certs



server.key

Clau privada del servidor ·

SECRET — protegir amb chmod 400



server.csr

Petició de signatura (CSR) · Només necessari durant la generació, es pot esborrar



server.crt

Certificat del servidor signat per la CA · El que configures a Apache/Nginx



certComplet.pem

(Opcional) Bundle complet per a serveis que ho requereixin

⚙️ BLOC 2 · VERIFICACIÓ

Ordres útils per comprovar que tot és correcte abans de desplegar

```
# Veure el contingut del certificat de la CA
openssl x509 -in rootCA.crt -text -noout

# Veure el contingut del certificat del servidor
openssl x509 -in server.crt -text -noout

# Verificar que server.crt és signat per rootCA.crt
openssl verify -CAfile rootCA.crt server.crt
server.crt: OK

# Verificar que la clau privada correspon al certificat
openssl rsa -in server.key -check
```



Tasca: Crear un entorn PKI complet

1. Crea un directori `~/practica-ssl` i treballa des d'aquí.
2. Genera la clau privada de la teva CA pròpia amb el nom **miCA.key**.
3. Crea el certificat arrel de la teva CA (**miCA.crt**) amb 730 dies de validesa. Posa el teu nom com a *Organization* i "La Meva CA - FP Seguretat" com a *Common Name*.
4. Genera la clau privada d'un servidor web fictici (**webserver.key**).
5. Crea el CSR per a aquest servidor (**webserver.csr**). Usa `webserver.local` com a *Common Name*.
6. Signa el CSR amb la teva CA per obtenir el certificat del servidor (**webserver.crt**) amb 365 dies de validesa.
7. **Bonus:** Verifica amb `openssl verify` que la cadena de confiança és correcta.
8. **Extra:** Genera el bundle **fullWeb.pem** amb els tres fitxers.

BLOC 3 · ERRORS COMUNS



"unable to load CA Private Key"

La ruta al fitxer `rootCA.key` és incorrecta, o el fitxer no existeix. Usa `ls -la` per confirmar els fitxers presents.



Certificate mismatch / ERR_CERT_COMMON_NAME_INVALID

El **Common Name** del certificat no coincideix amb el domini al qual accedeixes. El CN ha de ser exactament el hostname o domini.



"verification failed" en verificar server.crt

El certificat no va ser signat per aquesta CA, o vas usar la CA incorrecta al flag `-CAfile`. Comprova que uses els fitxers de la mateixa CA.



El navegador continua mostrant "No segur"

El navegador no confia en la teva CA. Has d'importar `rootCA.crt` a l'emmagatzematge de certificats de confiança del sistema o del navegador.

✓ RESUM



Conceptes

SSL/TLS, PKI, CA, clau privada, clau pública, CSR, FQDN, certificat autofirmat.



Habilitats

Crear CA pròpia, generar claus RSA 4096, emetre i signar certificats amb OpenSSL.



Competències

Dissenyar infraestructura PKI bàsica, gestionar certificats per a serveis interns.

"La criptografia no et protegeix de tots els atacants, però sí dels més descurats — i això ja és molt."

`openssl genrsa`

`openssl req -x509`

`openssl x509 -req`

`openssl verify`